C语言核心知识点总结

一、 基础输入输出与数据类型

1. **格式控制符**：
   - `%d` 整数，`%c` 单字符，`%s` 字符串，`%p` 地址。
   - `%f` 用于 printf 输出 float 和 double。
   - `%lf` 用于 scanf 输入 double。
   - `%-3d` 表示左对齐，最小宽度3。
2. **getchar 与 putchar**：
   - 用于单字符输入输出。
   - 核心用途：清理 scanf 留下的回车符（如 `while(getchar() != '\n');`）。
3. **数据类型大小**：
   - int(4字节), char(1字节), double(8字节)。类型不匹配会导致内存错乱。

二、 控制流与逻辑陷阱

1. **switch-case 穿透**：
   - 匹配到入口后，若没有 break，会无视后续 case 条件，直接往下执行。
2. **短路求值 (`&&` 和 `||`)**：
   - 左边能决定结果，右边绝不执行。
   - 会导致右侧的赋值、自增操作被跳过。
3. **continue 与 break**：
   - `continue` 跳过本次循环，进入下一次。
   - `break` 直接结束当前整个循环。
   - 避坑：在 while 循环中，continue 前必须要有自增变量，否则死循环。
4. **三目运算符 `?:`**：
   - 格式：条件 ? 值1 : 值2。
   - 判断依据：非0即真。优先级低于比较运算符，建议加括号。

三、 数组与字符串

1. **单字符 vs 字符串**：
   - `char ch = 'A';` (单字符，1字节)
   - `char ch[] = "A";` (字符数组，2字节，自动补 '\0')
2. **字符串结束符 `\0`**：
   - `sizeof` 算总容量（包含\0），`strlen` 算实际长度（不包含\0）。
3. **数组长度计算**：
   - `sizeof(array) / sizeof(array[0])`（仅限于定义数组的当前作用域）。

四、 指针核心（最难啃的骨头）

1. **核心概念**：
   - `a` 是值（客人）；`&a` 是地址（门牌号）；`p` 是指针（房卡）。
2. **`*` 与 `&` 的使用**：
   - `&` 取地址，普通变量要加。

- `*` 在定义/声明时表示“这是指针”。
- `*` 在执行/使用时表示“解引用”（顺着房卡找房间）。

3. **指针与数组**：
   - 数组名 arr 本身是首元素地址（等价于 &arr[0]）。
   - `ptr[1]` 完全等价于 `*(ptr + 1)`。
4. **致命雷区**：
   - 野指针（未初始化）绝对不能解引用，会崩溃。
   - `NULL` 是安全占位符，不能往里写数据。
   - 打印地址必须用 `%p`，用 `%d` 会截断出乱码。

五、 函数与参数传递

1. **声明与定义**：
   - 定义在 main 之后，必须在 main 前声明。定义在前则无需声明。
2. **`main` 与 `void`**：
   - `main` 必须用 `int`，末尾 `return 0;`。
   - `void` 表示无返回值或无参数。
3. **值传递 vs 地址传递**：
   - 普通传参改变不了原变量。
   - 传指针可以真正改变外部变量。
4. **参数与局部变量**：
   - 括号里的参数是外部给的原材料。
   - 花括号里的局部变量是函数内部自己造的工具。

六、 调试与开发环境

1. **运行 vs 调试**：
   - 运行：黑盒测试，看终端输出，跑完即止。
   - 调试（F5+断点）：上帝视角，冻结程序，通过左侧变量面板查看内存中的所有值。
2. **避坑**：
   - VS Code 的 GDB 调试器不支持中文路径，必须将文件夹改为纯英文。